SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Brute-Force and Exhaustive-Search techniques – Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: 19-10-2026

Q1 – Sorting Under Different Data Conditions 6 Marks

Problem Context – E-Commerce Order Processing

An e-commerce fulfilment centre arranges customer orders according to a **dispatch priority score**.

The development team is considering:

Selection Sort | Bubble Sort | Insertion Sort

However, the incoming data may be randomly ordered, nearly sorted, already sorted or significantly disordered.

Your Challenge

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the **characteristics of the input data**.

Before implementation:

- Predict how the three algorithms will behave under different input arrangements.
- Explain the algorithmic reason for your prediction.

Design **two deliberately contrasting datasets** and **one additional edge-case dataset of your own** that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

- Number of comparisons
- Number of swaps/movements
- Execution time, where useful

Compare your prediction with the observed behaviour.

Design Decision

The fulfilment centre reports that approximately **95% of existing orders are already in the correct priority order**, with only small batches of new orders added periodically.

Determine which of the three algorithms you would recommend and defend your decision.

Evidence Expected

Reasoned prediction, three deliberately designed input conditions, implementation of the three sorting algorithms, operation counts, comparison of predicted and observed behaviour, and an evidence-based recommendation.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? 4 Marks

Problem Context – Aircraft Maintenance Tool Tracking

An aircraft maintenance facility maintains a temporary list of specialised tools currently issued to technicians.

Each record contains:

Tool ID | Technician ID | Issue Time | Work Bay

The list changes continuously as tools are issued and returned.

When a technician asks:

> "Who currently has Tool T-184?"

the system uses **Sequential Search**.

Your Challenge

Design experiments to investigate Sequential Search under:

- Successful searches at different positions
- Unsuccessful searches
- Different list sizes

Record the **number of comparisons** required.

Then answer:

> **Under what operating conditions is Sequential Search a reasonable engineering choice, and when would you stop recommending it?**

Consider both:

- Dataset size
- Frequency of search requests

Changed Condition

The organisation expands from one hangar to **20 maintenance facilities**, resulting in substantially more active records and lookup requests.

Re-evaluate your original recommendation.

Evidence Expected

Designed search experiments, comparison-count evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed operating condition.

Q3 – Engineer the Behaviour of Brute-Force String Matching 6 Marks

Problem Context – Cybersecurity Log Screening

A security operations centre searches large text-based system logs for known event signatures represented as character patterns.

The initial system uses **Brute-Force String Matching**.

Your Challenge

Instead of merely implementing the algorithm, investigate **how the structure of the text and pattern influences computational effort**.

Design:

1. One text-pattern pair in which the algorithm performs relatively little work.
2. One text-pattern pair deliberately engineered to cause a **large number of character comparisons**.
3. One additional test condition of your own choice.

Before executing each test:

- Predict the expected behaviour.
- Explain the algorithmic reason for your prediction.

Implement the algorithm and record the number of character comparisons.

Inquiry

Compare your prediction with the experimental result.

Explain:

- Why one input requires substantially more work than another
- Which characteristics of the text and pattern create repeated comparisons
- What this reveals about evaluating an algorithm using only its input size

Evidence Expected

Correct implementation, three deliberately designed test conditions, prediction before execution, character-comparison evidence, and a reasoned explanation of the observed differences.

Q4 – Closest Pair and the Cost of Examining Every Possibility 6 Marks

Problem Context – Drone Collision-Risk Monitoring

A logistics company operates delivery drones. At a particular instant, each drone reports its position:

$$
D_i=(x_i,y_i)
$$

The monitoring system must identify the **two drones closest to one another**.

Your Challenge

Design a **Brute-Force Closest-Pair** solution.

Before implementation, determine:

- Which candidate pairs must be examined
- How duplicate pair comparisons can be avoided
- How many unique pairs exist for (n) drones

Derive the number of candidate pairs mathematically.

Implement your solution and test it using **at least three meaningfully different input sizes**.

Record:

- Number of pairs examined
- Number of distance calculations
- Execution time, where useful

Compare the theoretical growth with experimental behaviour.

Scalability Inquiry

Define what you consider an **acceptable response time** for your experimental environment.

Using your evidence, determine approximately when the brute-force approach begins to become impractical.

Finally, identify **where the computational inefficiency originates** and describe what a redesigned strategy would need to avoid or reduce.

You are not required to implement an advanced Closest-Pair algorithm in this question.

Evidence Expected

Algorithm design, mathematical pair-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and reasoned identification of the limitation of the brute-force strategy.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle 7 Marks

Problem Context – Wildlife Reserve Boundary Monitoring

Monitoring stations are positioned throughout a wildlife reserve. Each station is represented by coordinates:

[
P_i=(x_i,y_i)
]

Conservation officers need to determine the **outer boundary formed by the monitoring stations**.

You are required to develop a **Brute-Force Convex Hull** solution.

Design Starting Point

You are given only the following geometric principle:

> **A line joining two candidate points may form part of the outer boundary if all remaining points lie on an appropriate single side of that line.**

You are **not provided with the complete algorithm**.

Your Challenge

Convert this geometric property into a computational procedure.

Determine:

- How two candidate points define a line
- How the position of another point relative to that line can be determined
- How the remaining points must be examined

- What condition identifies a potential hull edge
- How the final boundary will be constructed

Develop your own pseudocode before implementation.

Test Your Design

Test the algorithm using:

- A point set containing both boundary and interior points
- A second point configuration deliberately designed by you to challenge your algorithm

Visualise the resulting boundary.

Analyse how the computational work grows as the number of points increases.

Correctness Inquiry

Explain:

> **Why does your geometric test provide evidence that an accepted edge belongs to the outer boundary?**

Your explanation should be based on geometric reasoning rather than simply referring to the program output.

Evidence Expected

A student-derived geometric decision rule, pseudocode, implementation, deliberately designed test configurations, computational analysis, boundary visualisation and a reasoned correctness argument.

Q6 – Exhaustive Search and Combinatorial Growth 8 Marks

Problem Context – Humanitarian Airlift Planning

A helicopter carrying emergency supplies has a strict payload limit.

Each available package contains:

Package ID | Weight | Humanitarian Value

The objective is to select a combination of packages that:

- Does not exceed the payload limit, and
- Maximises the total humanitarian value delivered.

Your Challenge

Design an **Exhaustive-Search strategy**.

Determine:

- How candidate package combinations will be represented
- How candidate solutions will be generated
- How feasibility will be checked
- How feasible solutions will be evaluated
- How the best solution will be retained

Do not begin from supplied pseudocode.

Correctness Check

Create a small instance for which the possible combinations can be independently verified.

Use this to establish whether your algorithm identifies the correct optimum.

Growth Investigation

Investigate increasing problem sizes.

Before experimentation:

- Derive/predict how the number of candidate combinations grows with (n).
- Predict what will happen as additional packages are introduced.

Use **at least three meaningful problem sizes** to validate your prediction experimentally.

Inquiry

Explain:

Why can adding what appears to be a relatively small number of additional packages produce a dramatic increase in computational effort?

Your explanation must relate to the growth of the **solution space**.

Deployment Decision

Assume that the airlift planner must produce a loading decision within a fixed response time selected and justified by you.

Using your experimental evidence, estimate the largest problem size for which you would be willing to deploy your exhaustive-search solution.

Evidence Expected

Student-designed exhaustive enumeration, correctness verification, mathematical prediction of solution-space growth, experimental validation, explanation of combinatorial growth and an evidence-based deployment boundary.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate — 13 Marks

Review the evidence you generated while working on Q1–Q6.

Select **ONE solution** that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are **not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better**.

Your task is to understand the computational problem first and then develop an appropriate redesign.

A. Diagnose the Computational Bottleneck — 2 Marks

Using your own mathematical and/or experimental evidence, identify:

- Where most computational effort is being spent
- Which operation grows significantly as the input increases
- Whether work is being unnecessarily repeated
- Which characteristic of the original algorithm causes the limitation

Evidence Expected

A precise diagnosis supported by evidence from your original implementation and analysis.

B. Define the Design Objective — 2 Marks

Before proposing a new solution, clearly state what your redesign must achieve.

For example:

- Reduce unnecessary comparisons
- Avoid examining every candidate
- Exploit existing ordering
- Reuse previously computed information
- Reduce the number of candidate solutions
- Reduce repeated processing

Specify a **measurable objective** wherever possible.

Evidence Expected

A clearly stated redesign objective derived directly from the identified computational limitation.

C. Design the Revised Computational Strategy — 4 Marks

Develop a revised solution.

Explain:

- What you have changed
- What information or property your new design exploits
- Which operations are avoided or reduced
- Whether preprocessing is introduced

- Whether additional memory is required
- Any new trade-offs created by the redesign

Provide appropriate pseudocode or a design representation.

Your marks will be based on the **quality of the algorithmic reasoning**, not merely on whether you name a known algorithm.

Evidence Expected

A coherent redesigned computational strategy directly addressing the diagnosed bottleneck, supported by appropriate design representation and technical justification.

D. Predict Before Implementing — 1 Mark

Before testing the redesigned solution:

- Predict how its behaviour should differ from the original.
- Identify the performance measure in which you expect improvement.

Evidence Expected

A clear, testable prediction made before experimental validation.

E. Implement and Validate the Redesign — 2 Marks

Implement a prototype of the redesigned strategy wherever feasible.

Using the **same or equivalent test conditions**, compare:

Original Solution → Redesigned Solution

Use appropriate measures such as:

- Comparisons
- Key operations
- Candidate solutions examined
- Distance calculations
- Character comparisons
- Execution time, where relevant

Evidence Expected

Comparable experimental evidence showing the behaviour of the original and redesigned solutions.

F. Defend the Final Design — 2 Marks

Evaluate whether the redesign achieved its intended objective.

Discuss:

- Whether the prediction was supported
- Magnitude of improvement, if any
- New costs or trade-offs
- Situations in which the original solution may still be preferable
- Whether the redesigned solution sufficiently addresses the identified limitation

A redesign that does **not** outperform the original will not automatically be considered unsuccessful if you correctly analyse and explain why the design hypothesis did not produce the expected result.

Evidence Expected

A critical, evidence-based evaluation of the redesigned solution and a defensible final engineering conclusion.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Selection, Bubble & Insertion Sort	**6**
Q2	Sequential Search	**4**
Q3	Brute-Force String Matching	**6**
Q4	Brute-Force Closest Pair	**6**
Q5	Brute-Force Convex Hull	**7**
Q6	Exhaustive Search	**8**

Q7	**Day-4 Design Challenge – Diagnose, Redesign & Validate**	**13**
	TOTAL	**50**

Submission Requirements

Submit:

- Your assignment report containing the required evidence for Q1–Q7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts, comparison counts or other algorithm-specific measures should be treated as **primary experimental evidence**. Execution time may be reported as additional evidence but should not be used as the sole basis for your conclusions.

Q1. SORTING UNDER DIFFERENT DATA CONDITIONS

Aim

To implement Selection Sort, Bubble Sort, and Insertion Sort, compare their performance for different input arrangements, and recommend the best algorithm for an e-commerce order-processing system.

Theory

1. Selection Sort: Finds the minimum element from the unsorted portion and swaps it with the first unsorted element.

- Best, average, and worst time complexity: $O(n^2)$.
- Space complexity: $O(1)$.
- Advantage: Performs relatively few swaps.
- Limitation: Always performs $n(n-1)/2$ comparisons.

2. Bubble Sort: Compares adjacent elements and swaps them if they are in the wrong order. An optimized version stops when a complete pass makes no swaps.

- Best case: $O(n)$.
- Average and worst cases: $O(n^2)$.
- Space complexity: $O(1)$.

3. Insertion Sort: Takes each element and inserts it into its correct position in the already-sorted portion.

- Best case: $O(n)$.
- Average and worst cases: $O(n^2)$.
- Space complexity: $O(1)$.
- Advantage: Efficient for nearly sorted data.

Predictions

Input condition	Selection Sort	Bubble Sort	Insertion Sort
Already sorted	$O(n^2)$ comparisons	$O(n)$	$O(n)$
Nearly sorted	$O(n^2)$ comparisons	Often few swaps	Usually efficient
Random	$O(n^2)$	$O(n^2)$	$O(n^2)$
Reverse sorted	$O(n^2)$	$O(n^2)$	$O(n^2)$

C Program

```
#include <stdio.h>
#define N 8

void selectionSort(int a[], int n, long *c, long *m) {
    int i, j, min, t;
    *c = *m = 0;
    for (i = 0; i < n-1; i++) {
        min = i;
        for (j = i+1; j < n; j++) {
            (*c)++;
            if (a[j] < a[min]) min = j;
        }
        if (min != i) {
            t=a[i]; a[i]=a[min]; a[min]=t;
            *m += 3;
        }
    }
}

void bubbleSort(int a[], int n, long *c, long *m) {
    int i, j, t, swapped;
    *c = *m = 0;
    for (i=0; i<n-1; i++) {
        swapped=0;
        for (j=0; j<n-i-1; j++) {
            (*c)++;
            if (a[j]>a[j+1]) {
                t=a[j]; a[j]=a[j+1]; a[j+1]=t;
                *m += 3;
                swapped=1;
            }
        }
        if (!swapped) break;
    }
}

void insertionSort(int a[], int n, long *c, long *m) {
    int i,j,key;
    *c = *m = 0;
    for (i=1; i<n; i++) {
        key=a[i]; (*m)++;
        j=i-1;
        while (j>=0) {
            (*c)++;
            if (a[j]>key) {
                a[j+1]=a[j]; (*m)++; j--;
            } else break;
        }
        a[j+1]=key; (*m)++;
    }
}
```

```
void display(int a[], int n) {
    int i;
    for (i=0;i<n;i++) printf("%d ",a[i]);
    printf("\n");
}

int main(void) {
    int data[3][N] = {
        {1,2,3,4,5,6,7,8},
        {1,2,3,5,4,6,7,8},
        {8,7,6,5,4,3,2,1}
    };
    char *names[]={"Sorted","Nearly sorted","Reverse sorted"};
    int a[N],i,j;
    long c,m;

    for(i=0;i<3;i++) {
        printf("\n%s\n",names[i]);
        for(j=0;j<N;j++) a[j]=data[i][j];
        selectionSort(a,N,&c,&m);
        printf("Selection: comparisons=%ld, movements=%ld\n",c,m);
        display(a,N);

        for(j=0;j<N;j++) a[j]=data[i][j];
        bubbleSort(a,N,&c,&m);
        printf("Bubble: comparisons=%ld, movements=%ld\n",c,m);
        display(a,N);

        for(j=0;j<N;j++) a[j]=data[i][j];
        insertionSort(a,N,&c,&m);
        printf("Insertion: comparisons=%ld, movements=%ld\n",c,m);
        display(a,N);
    }
    return 0;
}
```

Operation Counts

For n = 8:

Algorithm	Input	Comparisons	Swaps/movements
Selection Sort	Sorted	28	0 swaps
Bubble Sort	Sorted	7	0 swaps
Insertion Sort	Sorted	7 key comparisons	14 assignments
Selection Sort	Reverse	28	4 swaps
Bubble Sort	Reverse	28	28 swaps
Insertion Sort	Reverse	28 key comparisons	42 assignments

Movement counts depend on the counting convention. The program counts three assignments per swap and counts key assignments and shifts for Insertion Sort.

Recommendation and Conclusion

Insertion Sort is recommended because approximately 95% of orders are already sorted and only small batches are added periodically. It efficiently inserts new elements into an existing sorted list. The experiment demonstrates that algorithm performance depends on the input arrangement.

Q2. WHEN IS SEQUENTIAL SEARCH APPROPRIATE?

Aim

To investigate Sequential Search for locating aircraft maintenance tools under different list sizes and search conditions.

Theory

Sequential Search examines each record until the target is found or the list ends.

- Best case: O(1).
- Average case: O(n).
- Worst case: O(n).
- Space complexity: O(1).

It is suitable for small, unsorted, frequently changing lists.

C Program

```
#include <stdio.h>

int sequentialSearch(int a[], int n, int key, int *c) {
    int i;
    *c=0;
    for(i=0;i<n;i++) {
        (*c)++;
        if(a[i]==key) return i;
    }
    return -1;
}

int main(void) {
    int tools[]={101,102,184,205,220};
    int key,pos,c,i,n=5;

    printf("Tool IDs: ");
    for(i=0;i<n;i++) printf("T-%d ",tools[i]);
```

```
    printf("\nEnter tool ID: ");
    scanf("%d",&key);

    pos=sequentialSearch(tools,n,key,&c);

    if(pos!=-1) printf("Tool found at position %d\n",pos+1);
    else printf("Tool not found\n");

    printf("Comparisons: %d\n",c);
    return 0;
}
```

Sample Output

```
Tool IDs: T-101 T-102 T-184 T-205 T-220
Enter tool ID: 184
Tool found at position 3
Comparisons: 3
```

Comparison Table

List size	First-position success	Middle-position success	Last-position success	Absent target
5	1	3	5	5
10	1	5	10	10
20	1	10	20	20

These counts assume the target is at the stated position. An absent target requires checking every element.

Engineering Decision

Sequential Search is appropriate when the list is small, searches are infrequent, and records change frequently. When the organisation expands to 20 facilities, the records and lookup requests may increase significantly. A database index or hash table is a better choice for frequent lookups by unique Tool ID.

Conclusion

Sequential Search is simple and memory-efficient but becomes slow for large datasets because it may examine every record.

Q3. BRUTE-FORCE STRING MATCHING

Aim

To investigate how text and pattern structure affect the number of character comparisons.

Theory

Brute-Force String Matching checks a pattern at every possible starting position in a text.

For text length n and pattern length m:

- Worst-case time: O(nm).
- Space complexity: O(1).
- Maximum alignments: n − m + 1.

Repeated prefixes can cause repeated comparisons.

Test Cases and Predictions

1. Text `HELLO`, pattern `HE`: match at index 0 using 2 comparisons.
2. Text `AAAAAAAAAA`, pattern `AAAAB`: 6 alignments, with 5 comparisons each, giving 30 comparisons.
3. Text `COMPUTERSCIENCE`, pattern `XYZ`: 13 alignments, each failing on the first character.

C Program

```
#include <stdio.h>
#include <string.h>

int bruteForceMatch(const char text[], const char pattern[],
                    long *c) {
    int n=strlen(text), m=strlen(pattern), i,j;
    *c=0;
    if(m==0) return 0;
    if(m>n) return -1;

    for(i=0;i<=n-m;i++) {
        for(j=0;j<m;j++) {
            (*c)++;
            if(text[i+j]!=pattern[j]) break;
        }
        if(j==m) return i;
    }
    return -1;
}

int main(void) {
    const char *texts[]={"HELLO","AAAAAAAAAA","COMPUTERSCIENCE"};
    const char *patterns[]={"HE","AAAAB","XYZ"};
    long c;
    int i,pos;

    for(i=0;i<3;i++) {
        pos=bruteForceMatch(texts[i],patterns[i],&c);
        printf("\nText: %s\nPattern: %s\n",texts[i],patterns[i]);
```

```
        if(pos>=0) printf("Found at index %d\n",pos);
        else printf("Pattern not found\n");
        printf("Character comparisons: %ld\n",c);
    }
    return 0;
}
```

Sample Output

```
Text: HELLO
Pattern: HE
Found at index 0
Character comparisons: 2

Text: AAAAAAAAAA
Pattern: AAAAB
Pattern not found
Character comparisons: 30

Text: COMPUTERSCIENCE
Pattern: XYZ
Pattern not found
Character comparisons: 13
```

Analysis

Test	Text length	Pattern length	Comparisons
Immediate match	5	2	2
Repeated prefix	10	5	30
Pattern absent	15	3	13

The repeated-character case performs more work because the algorithm repeatedly matches four `A` characters before finding a mismatch. The absent-pattern case fails immediately at each alignment.

Input size alone does not determine exact computational effort; character arrangement also matters.

Conclusion

Brute-Force String Matching is simple but may repeat comparisons unnecessarily. KMP improves performance by reusing information about matched prefixes.

Q4. BRUTE-FORCE CLOSEST PAIR

Aim

To identify the two closest drones using their two-dimensional coordinates.

Theory

The distance between two points is:

$d = \sqrt{((x_2 - x_1)^2 + (y_2 - y_1)^2)}$

Every unique pair must be examined once. Using `j = i + 1` avoids duplicate pair comparisons.

The number of unique pairs is:

$n(n - 1)/2$.

Time complexity is $O(n^2)$; auxiliary space is $O(1)$.

C Program

```
#include <stdio.h>
#include <math.h>

typedef struct { double x,y; } Point;

void closestPair(Point p[],int n,int *first,int *second,
                 long *pairs,double *bestDistance) {
    int i,j;
    double dx,dy,d2,best2=INFINITY;

    *pairs=0;
    *first=*second=-1;

    for(i=0;i<n-1;i++) {
        for(j=i+1;j<n;j++) {
            dx=p[i].x-p[j].x;
            dy=p[i].y-p[j].y;
            d2=dx*dx+dy*dy;
            (*pairs)++;

            if(d2<best2) {
                best2=d2;
                *first=i;
                *second=j;
            }
        }
    }
    *bestDistance=sqrt(best2);
}

int main(void) {
    Point p[1000];
    int n,i,a,b;
    long pairs;
    double distance;

    printf("Enter number of drones (2-1000): ");
```

```
    scanf("%d",&n);
    if(n<2||n>1000) {
        printf("Invalid number of drones\n");
        return 1;
    }

    for(i=0;i<n;i++) {
        printf("Enter x and y for drone %d: ",i+1);
        scanf("%lf %lf",&p[i].x,&p[i].y);
    }

    closestPair(p,n,&a,&b,&pairs,&distance);
    printf("Closest drones: %d and %d\n",a+1,b+1);
    printf("Minimum distance: %.4f\n",distance);
    printf("Pairs examined: %ld\n",pairs);
    printf("Distance calculations: %ld\n",pairs);
    return 0;
}
```

Sample Input

```
4
0 0
3 4
1 1
2 2
```

Sample Output

```
Closest drones: 3 and 4
Minimum distance: 1.4142
Pairs examined: 6
Distance calculations: 6
```

Growth Table

Drones (n)	Unique pairs
10	45
100	4,950
1,000	499,500
10,000	49,995,000

Scalability Analysis

For this experiment, one second can be selected as an acceptable response-time target. The actual crossover point must be measured on the target computer.

The main inefficiency is examining every possible pair. Doubling the number of drones approximately quadruples the number of calculations. A divide-and-conquer closest-pair algorithm can reduce the standard time complexity to O(n log n).

Conclusion

Brute Force guarantees the closest pair for small inputs but becomes inefficient for large fleets because of quadratic growth.

Q5. BRUTE-FORCE CONVEX HULL

Aim

To find the outer boundary of a set of monitoring stations using a geometric test.

Theory

A convex hull is the smallest convex polygon containing all input points. Imagine stretching a rubber band around the outermost points.

For points A, B, and P, calculate:

$C = (Bx - Ax)(Py - Ay) - (By - Ay)(Px - Ax)$

- $C > 0$: P lies on one side of line AB.
- $C < 0$: P lies on the opposite side.
- $C = 0$: P is collinear with A and B.

If points lie strictly on both sides, AB cannot be a hull edge. A supporting line with all points on one side may define a boundary edge.

Pseudocode

```
FOR every pair of points A and B:
    positive = false
    negative = false

    FOR every other point P:
        calculate cross product C

        IF C > 0:
            positive = true
        ELSE IF C < 0:
            negative = true

    IF NOT (positive AND negative):
        AB is a supporting-line candidate

    Check collinear points to avoid redundant edge segments
    Add valid boundary edges to the hull
```

C Program

```
#include <stdio.h>
#define MAX 100

typedef struct { long long x,y; } Point;

long long cross(Point a,Point b,Point p) {
    return (b.x-a.x)*(p.y-a.y) -
           (b.y-a.y)*(p.x-a.x);
}

int onSegment(Point a,Point b,Point p) {
    return cross(a,b,p)==0 &&
        p.x >= (a.x<b.x?a.x:b.x) &&
        p.x <= (a.x>b.x?a.x:b.x) &&
        p.y >= (a.y<b.y?a.y:b.y) &&
        p.y <= (a.y>b.y?a.y:b.y);
}

int main(void) {
    Point p[MAX];
    int n,i,j,k,pos,neg,bad,mid,edges=0;

    printf("Enter number of points (3-100): ");
    scanf("%d",&n);
    if(n<3||n>MAX) {
        printf("Invalid number of points\n");
        return 1;
    }

    for(i=0;i<n;i++) {
        printf("Enter x and y for point %d: ",i+1);
        scanf("%lld %lld",&p[i].x,&p[i].y);
    }

    printf("\nConvex hull edges:\n");

    for(i=0;i<n;i++) {
        for(j=i+1;j<n;j++) {
            pos=neg=bad=mid=0;

            for(k=0;k<n;k++) {
                long long c;
                if(k==i||k==j) continue;
                c=cross(p[i],p[j],p[k]);

                if(c>0) pos=1;
                else if(c<0) neg=1;
                else if(!onSegment(p[i],p[j],p[k])) bad=1;
            }

            if(!(pos&&neg)&&!bad) {
                for(k=0;k<n;k++)
```

```
                    if(k!=i&&k!=j&&onSegment(p[i],p[j],p[k]))
                        mid=1;

                if(!mid) {
                    printf("(%lld,%lld) -- (%lld,%lld)\n",
                        p[i].x,p[i].y,p[j].x,p[j].y);
                    edges++;
                }
            }
        }
    }

    printf("Number of hull edges: %d\n",edges);
    return 0;
}
```

Test Case 1

Points:

(0,0), (4,0), (4,4), (0,4), (2,2), (1,1).

Expected boundary:

- (0,0) – (4,0)
- (4,0) – (4,4)
- (4,4) – (0,4)
- (0,4) – (0,0)

The points (2,2) and (1,1) lie inside the square.

Test Case 2

Points:

(0,0), (4,0), (2,3), (2,1), (1,1).

Expected boundary:

- (0,0) – (4,0)
- (0,0) – (2,3)
- (4,0) – (2,3)

The other two points are interior points.

Complexity

Number of points	Candidate pairs
10	45

50	1,225
100	4,950

There are $O(n^2)$ candidate pairs and up to n checks per pair, so time complexity is $O(n^3)$.

The cross-product test identifies whether all points lie on one side of a candidate line. Collinear point sets require special handling because their hull is a line segment rather than a polygon. The implementation also assumes distinct points and coordinates small enough to avoid integer overflow.

Conclusion

The Brute-Force Convex Hull method derives the boundary from geometric properties. It is useful for small datasets but inefficient for large point collections because its time complexity is cubic.

Q6. EXHAUSTIVE SEARCH AND COMBINATORIAL GROWTH

Aim

To select packages that maximise humanitarian value without exceeding a helicopter's payload capacity.

Theory

Each package can either be selected or excluded. Represent a subset using an n-bit binary mask:

- 1 means selected.
- 0 means excluded.

Each package has two choices, so n packages produce 2^n subsets.

For every subset:

1. Calculate total weight.
2. Calculate total value.
3. Reject it if the weight exceeds capacity.
4. Retain the feasible subset with the highest value.

Time complexity is $O(n \times 2^n)$.

C Program

```
#include <stdio.h>
#define MAX 25

int main(void) {
    int n,i,mask,capacity;
    int weight[MAX],value[MAX],bestMask=0,bestWeight=0;
    long long subsets,feasible=0,bestValue=0;

    printf("Enter number of packages (1-25): ");
    scanf("%d",&n);
    if(n<1||n>MAX) {
        printf("Invalid number of packages\n");
        return 1;
    }

    for(i=0;i<n;i++) {
        printf("Enter weight and value for package %d: ",i+1);
        scanf("%d %d",&weight[i],&value[i]);
        if(weight[i]<0||value[i]<0) {
            printf("Values must be non-negative\n");
            return 1;
        }
    }

    printf("Enter payload capacity: ");
    scanf("%d",&capacity);
    if(capacity<0) {
        printf("Invalid capacity\n");
        return 1;
    }

    subsets=1LL<<n;

    for(mask=0;mask<subsets;mask++) {
        long long v=0;
        int w=0;

        for(i=0;i<n;i++) {
            if(mask&(1<<i)) {
                w+=weight[i];
                v+=value[i];
            }
        }

        if(w<=capacity) {
            feasible++;
            if(v>bestValue) {
                bestValue=v;
                bestWeight=w;
                bestMask=mask;
            }
        }
```

```
    }

    printf("\nMaximum humanitarian value: %lld\n",bestValue);
    printf("Total weight: %d\n",bestWeight);
    printf("Selected packages: ");
    for(i=0;i<n;i++)
        if(bestMask&(1<<i)) printf("P%d ",i+1);

    printf("\nCandidate subsets examined: %lld\n",subsets);
    printf("Feasible subsets: %lld\n",feasible);
    return 0;
}
```

Correctness Example

Capacity = 5.

Package	Weight	Value
P1	2	30
P2	3	40
P3	4	50

Possible combinations:

Subset	Weight	Value	Feasible?
None	0	0	Yes
P1	2	30	Yes
P2	3	40	Yes
P3	4	50	Yes
P1 + P2	5	70	Yes
P1 + P3	6	80	No
P2 + P3	7	90	No
P1 + P2 + P3	9	120	No

The optimal solution is P1 + P2, with weight 5 and value 70.

Sample Output

```
Maximum humanitarian value: 70
Total weight: 5
Selected packages: P1 P2
Candidate subsets examined: 8
Feasible subsets: 5
```

Growth Investigation

Packages (n)	Candidate subsets (2^n)
5	32
10	1,024
15	32,768

20	1,048,576
25	33,554,432
30	1,073,741,824

Every additional package doubles the number of subsets. The nested package checks make the total work larger than the subset count alone.

Deployment Decision

Assume a one-second response-time target. Benchmark the program on the intended computer to determine the largest acceptable input. As a cautious starting point, exhaustive search is more suitable for small cases such as 15–20 packages. Larger cases may require dynamic programming or branch-and-bound.

Conclusion

Exhaustive Search guarantees the optimum because it examines every subset. Its limitation is exponential growth, which makes it unsuitable for large package collections.

Q7. DIAGNOSE, REDESIGN AND VALIDATE

Selected Problem: Brute-Force String Matching

The original algorithm repeatedly compares characters already matched in previous alignments. KMP redesigns the solution by reusing information about matched prefixes.

A. Diagnose the Bottleneck (2 Marks)

For text `AAAAAAAAAA` and pattern `AAAAB`, Brute Force matches four A characters at each alignment before encountering B. This work is repeated.

The worst-case time complexity is O(nm). The bottleneck is the repeated comparison of pattern prefixes.

B. Define the Design Objective (2 Marks)

The redesign should:

- Reduce repeated character comparisons.
- Preserve correct pattern detection.
- Improve worst-case time to O(n + m).
- Use a reasonable amount of extra memory.

The main measure is the number of character comparisons.

C. Revised Strategy (4 Marks)

KMP uses an LPS array: Longest Proper Prefix that is also a Suffix.

For the pattern `AAAAB`, the LPS array is:

```
Pattern: A A A A B
LPS:     0 1 2 3 0
```

When a mismatch occurs, KMP uses the LPS value to reuse previously matched information rather than restarting the pattern comparison.

- Preprocessing: O(m).
- Search: O(n).
- Total time: O(n + m).
- Extra space: O(m).

D. Prediction (1 Mark)

KMP should perform fewer comparisons on repeated-prefix patterns. Both algorithms should return the same match position or report no match.

E. Implement and Validate (2 Marks)

```
#include <stdio.h>
#include <string.h>
#define MAX 10000

int bruteForce(const char text[],const char pattern[],long long *c) {
    int n=strlen(text),m=strlen(pattern),i,j;
    *c=0;
    if(m==0) return 0;

    for(i=0;i<=n-m;i++) {
        for(j=0;j<m;j++) {
            (*c)++;
            if(text[i+j]!=pattern[j]) break;
        }
        if(j==m) return i;
    }
    return -1;
}

void buildLPS(const char p[],int m,int lps[]) {
    int len=0,i=1;
    lps[0]=0;

    while(i<m) {
        if(p[i]==p[len]) {
```

```
            len++;
            lps[i]=len;
            i++;
        } else if(len>0) {
            len=lps[len-1];
        } else {
            lps[i]=0;
            i++;
        }
    }
}

int kmpSearch(const char text[],const char pattern[],long long *c) {
    int n=strlen(text),m=strlen(pattern);
    int lps[MAX],i=0,j=0;
    *c=0;
    if(m==0) return 0;

    buildLPS(pattern,m,lps);

    while(i<n) {
        (*c)++;
        if(text[i]==pattern[j]) {
            i++;
            j++;
            if(j==m) return i-j;
        } else if(j>0) {
            j=lps[j-1];
        } else {
            i++;
        }
    }
    return -1;
}

int main(void) {
    char text[MAX],pattern[MAX];
    long long bc,kc;
    int bp,kp;

    printf("Enter text: ");
    if(scanf("%9999s",text)!=1) return 1;

    printf("Enter pattern: ");
    if(scanf("%9999s",pattern)!=1) return 1;

    bp=bruteForce(text,pattern,&bc);
    kp=kmpSearch(text,pattern,&kc);

    printf("\nBrute Force position: %d\n",bp);
    printf("Brute Force comparisons: %lld\n",bc);
    printf("\nKMP position: %d\n",kp);
    printf("KMP search comparisons: %lld\n",kc);

    if(bp==kp) printf("Validation successful: results agree.\n");
```

```
    else printf("Validation failed.\n");

    return 0;
}
```

Validation Test

Text: AAAAAAAAAAAAAAAAAAAA

Pattern: AAAAAB

Both algorithms should report no match. Run the program and record the comparison counts shown by your compiler.

Comparison Table

Measure	Brute Force	KMP
Preprocessing	None	O(m)
Worst-case time	O(nm)	O(n + m)
Extra space	O(1)	O(m)
Reuses matched prefixes	No	Yes
Best use	Small, simple searches	Larger searches with repeated prefixes

F. Defend the Final Design (2 Marks)

KMP addresses the bottleneck by avoiding unnecessary restarts. It should improve performance for repeated-prefix patterns while preserving correctness. Its trade-offs are preprocessing and extra memory. Brute Force may remain preferable for very small inputs because it is simpler.

Conclusion

The redesign demonstrates that understanding the source of repeated computation can lead to a more efficient solution. KMP is appropriate for large log searches where repeated character comparisons are expensive.

OVERALL CONCLUSION

This assignment demonstrates that algorithm performance depends on input arrangement, dataset size, frequency of operations, and response-time requirements.

1. Insertion Sort performs well on nearly sorted data.
2. Sequential Search is suitable for small, changing lists but becomes inefficient for large datasets.
3. Brute-Force String Matching may repeat character comparisons.

4. Brute-Force Closest Pair has quadratic growth.
5. Brute-Force Convex Hull has cubic growth.
6. Exhaustive Search guarantees an optimum but has exponential growth.
7. KMP improves string matching by reusing previously computed prefix information.

A good engineering solution must be correct, efficient, and supported by mathematical analysis, operation counts, and reproducible experiments.

REFERENCES

1. Anany Levitin, *Introduction to the Design and Analysis of Algorithms*, Pearson.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, MIT Press.
3. Donald E. Knuth, James H. Morris Jr., and Vaughan R. Pratt, “Fast Pattern Matching in Strings,” *SIAM Journal on Computing*, 1977.